

Esercizio di programmazione concorrente

Si consideri il codice dato, in cui compaiono dei thread Peer, ciascuno dotato di un DepositoDocumenti.

Ciascun Peer, iterativamente svolge la seguente sequenza di operazioni:

- 1) determina il titolo del documento che vuole usare;
- 2) consulta il catalogo per ottenere il riferimento al deposito in cui si trova il documento;
- 3) accede al documento: poiché non si vuole che lo stesso documento sia usato concorrentemente da più Peer, il documento viene prima occupato, poi usato e infine liberato per usi successivi.

Il Catalogo mantiene una tabella di corrispondenza tra titoli di Documenti e il DepositoDocumenti in cui si trovano. È responsabilità di ciascun Peer informare il Catalogo dei documenti disponibili nel deposito associato al Peer stesso.

Il codice dato non gestisce la sincronizzazione degli accessi ai documenti.

Si chiede di modificare il codice dato in modo che quando un Peer cerca di accedere a un documento occupato, venga messo in attesa fino a quando il documento non torna disponibile.

È necessario evitare i problemi tipici della programmazione concorrente (corse critiche, deadlock, ecc.)

Si ricorda che bisogna consegnare i file .java (non i .class!), raggruppati in un unico file zip.

Esercizio di programmazione distribuita (con uso di socket)

Si realizzi un sistema distribuito in cui c'è un unico Server e molti Client, potenzialmente residenti su macchine diverse.

Il Server ha la funzione un DepositoDocumenti centralizzato e accetta richieste delle seguenti operazioni:

- Aggiungere al deposito centrale un documento.
- Inizio accesso al documento avente un titolo dato
- Operazioni su un documento avente il titolo dato
- Fine accesso al documento avente il titolo dato

Ciascun documento può essere usato da un solo client alla volta.

Ciascun Client svolge iterativamente una delle operazioni seguenti:

- Crea un nuovo documento e lo manda al Server perché lo inserisca nel deposito centrale.
- Utilizzo di un documento esistente: inizio, operazioni, fine.

Si richiede di scrivere il codice del sistema descritto, usando i socket, evitando i problemi tipici della programmazione concorrente (corse critiche, deadlock ecc.). Si può riutilizzare parte del codice scritto per l'esercizio di programmazione concorrente.

Si ricorda che bisogna consegnare i file .java (non i .class!), raggruppati in un unico file zip.

Esercizio di programmazione distribuita (con uso di RMI)

Si desidera un sistema distribuito che funziona come il sistema descritto nell'esercizio di programmazione concorrente, ma in cui il catalogo e i peer possono risiedere su macchine diverse.

Si realizzi dunque un sistema distribuito in cui c'è un unico catalogo e molti Peer (ciascuno col suo deposito documenti). Il catalogo si comporta come un server che accetta richieste delle seguenti operazioni:

- Generazione di un identificatore univoco per i Peer.
- Aggiungere al Catalogo un record <titolo, riferimentoPeer>, dove riferimentoPeer è il riferimento al Peer che possiede il Deposito in cui si trova il documento avente il titolo indicato.
- Ricerca nel Catalogo del riferimento al Peer che possiede un titolo dato.

Ciascun Peer inizialmente informa il Catalogo dei titoli presenti nel proprio catalogo, dopo di che, concorrentemente:

- Iterativamente, determina il titolo del documento che vuole usare, consulta il catalogo per ottenere il riferimento al Peer nel cui deposito si trova il documento, e infine accede al documento. L'accesso prevede (come nel caso dell'esercizio sulla concorrenza) di occupare il documento, poi usarlo e infine liberarlo per usi successivi.
- Risponde alle richieste degli altri Peer:
 - inizio utilizzo del documento avente il titolo dato
 - uso del documento avente il titolo dato
 - fine utilizzo del documento avente il titolo dato

Si noti che i Peer si comportano da client nei confronti del catalogo e da server nei confronti degli altri Peer. Quest'ultima caratteristica è dovuta al fatto che un Peer A non può accedere direttamente ai dati di un Peer B: deve usare le operazioni apposite (occupa, usa e libera) fornite dal Peer B.

Si richiede di scrivere il codice del sistema descritto, usando RMI ed evitando i problemi tipici della programmazione concorrente (corse critiche, deadlock, ecc.)

Non si chiede di gestire la terminazione dei Peer.

È possibile riusare parte del codice prodotto per l'esercizio di programmazione concorrente.

Si ricorda che bisogna consegnare i file .java (non i .class!), raggruppati in un unico file zip.